

8-Puzzle Interactivo con Heurísticas

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Problema	2
2. Representación del Estado	2
3. Heurísticas Implementadas	2
3.1. Distancia Manhattan	2
3.2. Piezas Mal Colocadas	2
4. Algoritmo de Búsqueda Heurística	2
5. Cálculo de la Heurística Manhattan	3
6. Generación de Movimientos	3
7. Interfaz Gráfica	4

1. Descripción del Problema

El **8-Puzzle** consiste en un tablero de 3×3 con 8 fichas numeradas del 1 al 8 y un espacio vacío. El objetivo es alcanzar el estado meta deslizando fichas al espacio vacío.

$$\text{Estado meta: } \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & - \end{bmatrix}$$

2. Representación del Estado

El tablero se representa como un arreglo unidimensional de 9 enteros, donde el valor 0 representa el espacio vacío. Los movimientos legales desplazan el espacio vacío en una de las 4 direcciones (arriba, abajo, izquierda, derecha).

3. Heurísticas Implementadas

3.1. Distancia Manhattan

Suma de distancias horizontal y vertical de cada ficha hasta su posición meta:

$$h_M(n) = \sum_{i=1}^8 (|fila_i - fila_i^*| + |col_i - col_i^*|)$$

Esta heurística es **admisible** (nunca sobreestima) y **consistente**, lo que garantiza que la búsqueda encuentre la solución óptima.

3.2. Piezas Mal Colocadas

Cuenta el número de fichas que no están en su posición correcta:

$$h_P(n) = |\{i : \text{tablero}[i] \neq \text{meta}[i], \text{tablero}[i] \neq 0\}|$$

Es menos informada que Manhattan, por lo que genera más nodos antes de encontrar la solución.

4. Algoritmo de Búsqueda Heurística

Se implementa una variante de **Best-First Search** usando una cola de prioridad ordenada por el valor heurístico $h(n)$:

```
1 Nodo_BFS resolver(int heuristica) {
2     PriorityQueue<Nodo_BFS> cola =
3         new PriorityQueue<>(Comparator.comparingInt(
4             n -> calcularH(n.getEstado(), heuristica)));
5     cola.add(estadoInicial);
6     visitados.add(estadoInicial);
7 }
```

```

8     while (!cola.isEmpty()) {
9         Nodo_BFS actual = cola.poll();
10        if (esMeta(actual)) return actual;
11
12        for (Nodo_BFS hijo : generarHijos(actual)) {
13            if (!estaVisitado(hijo)) {
14                visitados.add(hijo);
15                cola.add(hijo);
16            }
17        }
18    }
19    return null;
20 }

```

5. Cálculo de la Heurística Manhattan

```

1  int manhattan(int[] tablero) {
2      int total = 0;
3      for (int i = 0; i < 9; i++) {
4          if (tablero[i] == 0) continue;
5          int valor = tablero[i] - 1;
6          int filaActual = i / 3;
7          int colActual = i % 3;
8          int filaMeta = valor / 3;
9          int colMeta = valor % 3;
10         total += Math.abs(filaActual - filaMeta)
11             + Math.abs(colActual - colMeta);
12     }
13     return total;
14 }

```

6. Generación de Movimientos

Los movimientos válidos se determinan por la posición del espacio vacío:

```

1  List<int[]> generarHijos(int[] estado) {
2      int pos = posicionVacio(estado); // ndice del 0
3      int fila = pos / 3, col = pos % 3;
4      int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};
5      List<int[]> hijos = new ArrayList<>();
6
7      for (int[] d : dirs) {
8          int nf = fila + d[0], nc = col + d[1];
9          if (nf >= 0 && nf < 3 && nc >= 0 && nc < 3) {
10             int[] hijo = estado.clone();
11             int npos = nf*3 + nc;
12             hijo[pos] = hijo[npos];
13             hijo[npos] = 0;
14             hijos.add(hijo);

```

```
15         }
16     }
17     return hijos;
18 }
```

7. Interfaz Gráfica

- El tablero se dibuja con `JButton[9]` en un `GridLayout(3,3)`.
- El botón **Mezclar** aplica 50 movimientos aleatorios válidos.
- Al presionar **Resolver**, se ejecuta el algoritmo y se anima la solución paso a paso.
- El contador muestra los pasos de la solución encontrada.